

Excellent! Now let's learn the **OSI model like an engineer**, not like someone memorizing interview answers.

We'll cover only the **top three layers** in this lesson.

Application → Presentation → Session

We'll follow one example throughout:

You open Chrome and type:

`https://www.google.com`

Big Picture

Before pressing Enter

Human

↓

Browser

↓

Operating System

↓

Network

After pressing Enter

You

|

| Click Enter

▼

Browser

|

▼

Application Layer



Notice:

The browser **does NOT directly send electrical signals**.

It simply creates **data**.

Layer 7 — Application Layer

What is the Application Layer?

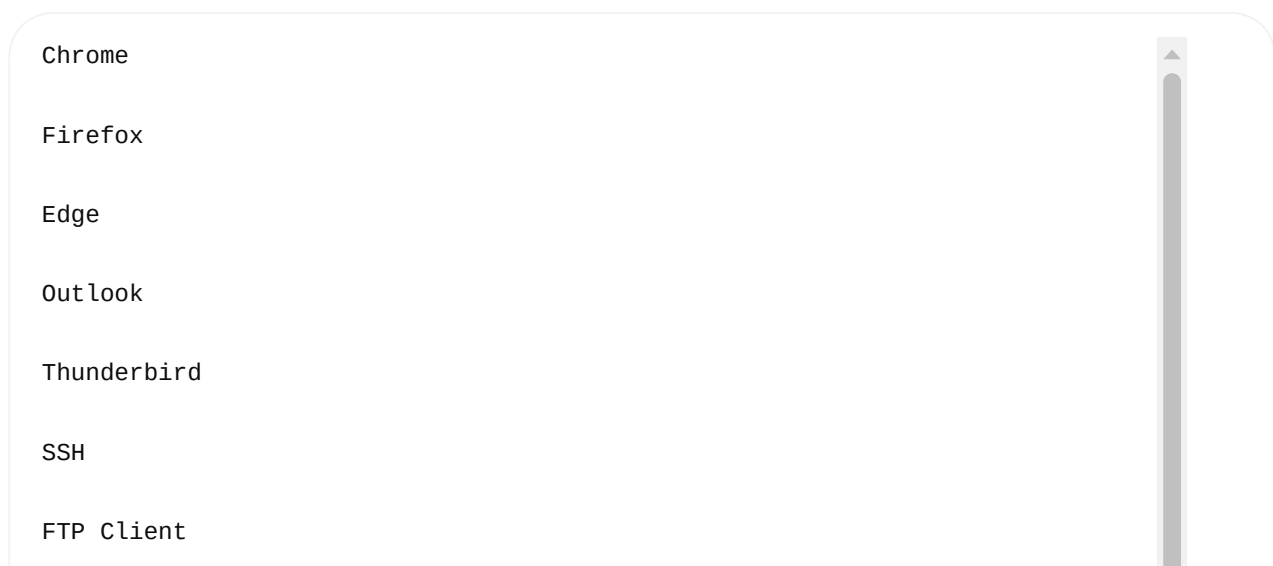
This is the layer **closest to the user**.

It provides networking services to applications.

It **does NOT mean the application itself**.

Instead, it means the networking functionality used by applications.

Examples



```
curl
```

```
wget
```

These applications generate network requests.

Real Example

Suppose you type

```
https://www.google.com
```

Then press Enter.

Question

What does Chrome actually create?

It creates an HTTP request.

Example

```
GET / HTTP/1.1
Host: www.google.com

User-Agent: Chrome

Accept: text/html
```

This is simply text.

Nothing more.

No IP.

No MAC.

No TCP.

Only application data.

Visualization

```
+-----+
```

GET / HTTP/1.1

Host: www.google.com

User-Agent: Chrome

Accept: text/html

+-----+

What Happens Inside Chrome?

Let's zoom in.

You press Enter

↓

Chrome checks URL

↓

Extracts Protocol

HTTPS

↓

Extracts Domain

www.google.com

↓

Creates HTTP Request

↓

Passes Data to OS

Notice something.

Chrome does **NOT** know

- IP Address
- MAC Address
- Ethernet
- Wi-Fi

Chrome only knows

"I want this webpage."

Responsibilities of Application Layer

The Application Layer is responsible for

- ✓ Creating requests
- ✓ Receiving responses
- ✓ Choosing protocol
- ✓ Interacting with user

Common Application Protocols

Protocol	Purpose	Default Port
HTTP	Web Browsing	80
HTTPS	Secure Web	443
FTP	File Transfer	21
SSH	Remote Login	22
SMTP	Send Email	25
IMAP	Read Email	143
POP3	Receive Email	110
DNS	Name Resolution	53
DHCP	Get IP Address	67/68

Interview Question

Does HTTP know IP Address?

Answer

No.

HTTP simply creates the request.

The lower layers handle addressing and delivery.

Layer 6 — Presentation Layer

Now the request reaches Layer 6.

Why Do We Need Presentation Layer?

Imagine two computers.

Computer A stores text as

UTF-8

Computer B stores text as

ASCII

Without conversion

They cannot understand each other.

Presentation Layer solves this.

Responsibilities

- ✓ Data Formatting
 - ✓ Character Encoding
 - ✓ Encryption
 - ✓ Compression
-

Visualization

Application Data





Data Formatting

Suppose Chrome creates

Hello

Memory

H

e

l

l

o

Presentation Layer converts this into a standard representation.

Examples

ASCII

UTF-8

UTF-16

Unicode

Example

Character

A

ASCII

65

Binary

01000001

Now every computer understands it.

Compression

Suppose

AAAAAAAAAAAAAAAAAAAA

Instead of sending

20 characters

Compression becomes

20 × A

Much smaller.

Examples

ZIP

GZIP

LZ4

Why?

Less bandwidth

Faster transmission

Encryption

Now the important part.

You opened

`https://google.com`

Notice

HTTPS

NOT

HTTP

Presentation Layer encrypts the data.

Before encryption

`GET / HTTP/1.1`

Anyone capturing packets can read it.

After encryption

`KJH78df89HGS98d...`

`AJHG87sdf987...`

Nobody can understand it without the key.

Visualization

HTTP Request

↓

Encryption

↓

Encrypted Data



Transport Layer



Interview Question

Which layer performs encryption?

Traditional OSI answer

Presentation Layer

Practical Linux answer

TLS is implemented in user-space libraries (OpenSSL, GnuTLS, etc.) and is often associated with the Presentation Layer conceptually.

Layer 5 — Session Layer

Now we have encrypted data.

Question

How does Google know

You

are the same person after clicking

10 pages?

Session Layer.

Responsibilities

- ✓ Establish Session
- ✓ Maintain Session
- ✓ Synchronization
- ✓ Close Session

Visualization

Laptop

↓

Create Session

↓

Communicate

↓

Maintain Session

↓

Close Session

Login Example

Suppose

You login to Gmail.

Username

Password

↓

Server

↓

Login Successful

Google creates

Session ID

XYZ12345

Every future request contains

Session ID

XYZ12345

Google knows

"This is the same user."

Visualization

Request 1

↓

Session Created

↓

Request 2

↓

Same Session

↓

Request 3

↓

Same Session

↓

Logout

↓

Session Destroyed

Another Example

SSH

ssh user@server

Connection established.

Now

You type

```
ls
```

Then

```
pwd
```

Then

```
cd /home
```

All commands use the same SSH session until you disconnect.

Synchronization

Imagine transferring a

2 GB file.

Connection breaks at

1.8 GB.

Should the transfer restart from

0 GB?

No.

Session Layer allows synchronization points (checkpoints), so communication can resume efficiently rather than always starting over.

Complete Flow So Far

You type

https://www.google.com

↓

Chrome

↓

Creates HTTP Request

↓

Presentation Layer

Encrypts

Formats

Compresses

↓

Session Layer

Creates Session

↓

Transport Layer

What Information Exists After Layer 5?

At this point, the data still **does not have**:

× IP Address

× MAC Address

× TCP Header

× Ethernet Frame

It only contains

+-----+



Encrypted HTTP Request

+-----+

Session Information

+-----+

The next layer (Transport) will add the first networking header.

Linux Examples

Application Layer

Open a webpage

```
curl https://www.google.com
```

Open SSH

```
ssh user@192.168.1.100
```

DNS lookup

```
dig google.com
```

Presentation Layer

View a TLS certificate

```
openssl s_client -connect google.com:443
```

This lets you observe the TLS handshake and the server's certificate.

Session Layer

Open an SSH session

```
ssh user@192.168.1.100
```

Run multiple commands

```
pwd  
ls  
cd /tmp
```

All commands are executed within the same SSH session until it ends.

Interview Questions

1. Is Chrome part of the OSI model?

No. Chrome is an application that uses the networking services represented by the Application layer.

2. Which layer creates the HTTP request?

The Application layer.

3. Which layer performs encryption?

Conceptually, the Presentation layer. In real Linux systems, TLS encryption is commonly implemented by user-space libraries such as OpenSSL.

4. Which layer creates sessions?

The Session layer is responsible for establishing, maintaining, and terminating communication sessions.

5. Does the Application layer know the destination IP address?

Typically, no. It knows the hostname (for example, `www.google.com`). DNS resolution and the lower networking stack determine the IP address before the packet is transmitted.

Visual Memory

```
Application
|
| "What do I want to send?"
▼
Presentation
|
| "How should I format, compress,
|   and protect the data?"
▼
Session
|
| "Who am I communicating with,
|   and is this conversation still active?"
▼
Transport
|
| "Now I'll prepare it for reliable delivery."
```

What's Next (Part 3)

In the next lesson, we'll study the **Transport Layer** in depth. This is one of the most important topics for C/C++ and Embedded Linux interviews because it directly relates to **socket programming**.

You'll learn:

- Why TCP and UDP exist
- How `socket()`, `bind()`, `listen()`, `accept()`, and `connect()` fit into the OSI model
- TCP three-way handshake with packet-by-packet diagrams
- TCP header fields (Source Port, Destination Port, Sequence Number, ACK Number, Window Size, Flags)
- Segmentation and reassembly
- Flow control and basic congestion control
- How a `send()` call in your C program eventually becomes TCP segments on the network

This will bridge the gap between networking theory and the Linux socket APIs you'll use in real applications.

